SIMATS ENGINEERING

ASSESSMENT TOOL 3: Mock Interview

Course Name	Computer Vision with OpenCV
Course Code	DSA02
Course Outcome	CO5: Understand video processing – motion computation – 3D vision – geometry (BTL 6)
Name and Reg No	V M Sai Bhargav - 192324126
Questions Answered	Q1, Q12, and Q56

Question 1 (10 Marks): OpenCV Pipeline for Reading and Processing Multiple Images from a Folder

Interviewer: *Design and explain a complete OpenCV pipeline to read, process, and display multiple images from a folder efficiently. Clearly justify your design choices and discuss potential challenges and optimizations.*

Candidate's Response:

I'd break this into four stages: file discovery, batch loading, processing, and output — kept as separate functions so each can be tuned independently.

First, for file discovery, I'd use Python's glob or os.listdir() to collect all image paths from the folder, filtering by extension (.jpg, .png, .bmp) so non-image files don't crash the loop. Second, for loading, rather than loading every image into memory at once, I'd loop through paths and read them one at a time with cv2.imread(), processing and releasing each before moving to the next — this keeps memory usage flat regardless of folder size. Third, in the processing stage, I'd apply the required operations — resizing, grayscale conversion, or filtering — inside a single reusable function so the same logic applies uniformly to every image. Finally, for output, I'd either display each result briefly with cv2.imshow() plus cv2.waitKey(), or, for a batch job, write results directly to an output folder with cv2.imwrite() and skip live display altogether.

The main design decision here is streaming one image at a time instead of loading the whole folder into a list — that avoids memory blowing up on large datasets. For optimization, I'd also add a check with os.path.exists() and a try/except around cv2.imread() to skip corrupted files gracefully rather than letting one bad file stop the whole batch. If performance mattered more, I'd look at multiprocessing to process several images in parallel, since image operations here are independent of each other.

Question 12 (10 Marks): Reducing False Positives in Face Detection

Interviewer: *A face detection system produces multiple false positives. Analyze the issue and propose improvements, explaining how each change enhances accuracy.*

Candidate's Response:

False positives in something like a Haar Cascade detector usually come from three places: detection parameters that are too permissive, poor image preprocessing, or background regions that just happen to resemble facial patterns.

My first step would be tuning the detectMultiScale() parameters — specifically increasing minNeighbors, since a higher value forces the detector to require more overlapping candidate detections before confirming a face, which cuts down on stray single detections. I'd also constrain minSize and maxSize to a realistic range for the expected face size in the frame, so tiny background artifacts or oversized regions get rejected outright. Second, on the preprocessing side, I'd make sure the image is properly grayscale-converted and contrast-normalized with something like CLAHE, because a cascade trained on well-lit, well-contrasted faces performs worse on noisy or poorly-lit frames, and that noise is a common source of spurious detections. Third, if false positives are still too frequent, I'd consider upgrading from a Haar cascade to a DNN-based face detector (OpenCV's res10 SSD model, for example) — it's more robust because it's learned discriminative features rather than relying on simple contrast patterns, at the cost of a bit more compute.

Each of these changes targets a different source of error: minNeighbors and size constraints filter at the detection-logic level, preprocessing improves the quality of what the detector even sees, and switching detector architecture raises the ceiling on accuracy itself. I'd apply them in that order, since the first two are essentially free in terms of added computation.

Question 56 (10 Marks): Real-Time Attendance System Using Face Recognition

> ***Interviewer:*** *Design a real-time attendance system using face recognition. Explain the pipeline and justify your design.*

Candidate's Response:

I'd design this as three phases: enrollment, real-time recognition, and attendance logging.

In the enrollment phase, I'd capture a handful of reference images per registered person using the webcam, detect and crop the face region, and store those samples labeled with the person's ID. I'd train an LBPH face recognizer (cv2.face.LBPHFaceRecognizer_create()) on this dataset, since it's lightweight and works well on modest hardware, which matters if this is deployed on a classroom or office PC rather than a server. In the real-time recognition phase, the system continuously reads frames from the webcam, converts each to grayscale, runs Haar cascade face detection, and for every detected face, passes the cropped region to the recognizer, which returns a predicted label and a confidence score. I'd only accept a match if the confidence score is below a set threshold — otherwise I'd mark the person as unrecognized, since a low-confidence match is more likely to be wrong than right.

For the attendance logging stage, I'd maintain an in-memory set of already-marked IDs for the current session, and only write a new row (name, timestamp) to a CSV or database the first time that ID is confidently recognized — this prevents the same person from being logged repeatedly every frame they remain in view. I'd also add a short cooldown or the marked-set check specifically to avoid duplicate entries, which is the most obvious failure mode in a naive version of this system.

I chose LBPH over a deep embedding-based approach (like FaceNet) because for a small, fixed set of registered users — say a classroom of 60 students — LBPH's speed and low resource requirement outweigh the small accuracy gap, and it avoids needing a GPU. The design decisions throughout — thresholded confidence, once-per-session marking, and a lightweight recognizer — are all aimed at making the system dependable and false-positive-resistant rather than just fast.

Declaration: *I certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*